

操作系统内核设计

Undefined OS

清华大学

徐子航 宋建昊 黄雨婕

2025 年 8 月

目录

第 1 章	摘要	1
	1.1 Undefined OS 内核介绍	1
	1.2 Undefined OS 内核整体设计	1
	1.3 实现参考与致谢	2
第 2 章	系统概述	3
	2.1 整体架构	3
	2.2 代码结构	3
	2.3 组件化的设计理念	4
	2.4 系统调用接口设计	5
第 3 章	开发板适配	8
	3.1 RISC-V 昉·星光 2	8
	3.1.1 执行流程	8
	3.1.2 内存地址映射	9
	3.1.3 页表结构与设置	9
	3.1.4 关于固件	10
	3.2 loongarch-龙芯 2k1000	10
第 4 章	应用程序支持	11
	4.1 发行版支持	11
	4.2 集成开发环境支持	11
	4.3 已支持应用程序列表	11
	4.3.1 BusyBox	11
	4.3.2 编译器支持 (Tcc、gcc、g++、rustc)	12
	4.3.3 git	12
	4.3.4 BusyBox vi	13
	4.3.5 grep	13
	4.3.6 lua	13
	4.3.7 python	13
	4.3.8 SQLite3	14
第 5 章	图形界面 (GUI) 支持	15
	5.1 GUI 架构	15

	5.2	渲染支持	15
	5.3	输入系统支持	16
	5.4	X11 协议支持	16
	5.5	Xfce 桌面支持	16
第 6 章		内存管理模块	18
	6.1	内存布局	18
	6.2	物理内存分配	18
	6.2.1	堆管理	19
	6.3	虚拟内存管理	19
	6.3.1	虚拟内存机制	19
	6.3.2	虚拟内存数据结构	20
	6.3.3	页表管理	20
	6.4	共享内存管理	21
	6.4.1	共享内存的生命周期管理	21
	6.4.2	进程级映射的优化设计	22
	6.4.3	持久性与泄漏防范的平衡	22
	6.4.4	共享内存设计总结	22
	6.5	优化设计	23
	6.5.1	写时复制(CoW)	23
	6.5.2	懒分配(Lazy Allocation)	26
第 7 章		进程管理模块	29
	7.1	核心数据结构	29
	7.1.1	线程(Thread)	29
	7.1.2	进程 (Process)	29
	7.1.3	进程组(ProcessGroup)	32
	7.1.4	会话(Session)	33
	7.2	PID/TID 管理	34
	7.3	层次结构	35
	7.4	进程创建与 fork 实现	35
	7.5	生命周期管理	36
	7.5.1	线程创建	37
	7.5.2	线程退出	38

	7.5.3	进程退出	39
	7.5.4	进程组创建/迁移	40
	7.6	数据分离设计	40
	7.7	资源限制	41
	7.8	进程管理总结	42
第 8 章		文件系统模块	43
	8.1	虚拟文件系统架构	43
	8.2	文件系统抽象层	44
	8.3	索引节点抽象层	45
	8.4	目录项抽象层	46
	8.5	文件对象抽象层	47
	8.6	进程间通信管道	48
	8.7	文件系统总结	49
第 9 章		信号处理模块	50
	9.1	信号的发送	50
	9.2	信号处理时机	51
	9.3	信号处理与退出状态	51
第 10 章		网络模块	52
	10.1	架构设计	52
	10.1.1	底层网络系统	52
	10.1.1.1	LoopbackDev 实现	52
	10.1.1.2	设备抽象 (Device)	53
	10.1.1.3	监听与连接管理	54
	10.1.1.4	地址与路由管理	54
	10.1.1.5	DNS 查询	54
	10.1.2	TCP	54
	10.1.3	UDP	55
第 11 章		总结与展望	56

第 1 章 摘要

1.1 Undefined OS 内核介绍

Undefined OS 是一个用 Rust 语言编写的操作系统内核，支持多种体系结构（包括 riscv64、aarch64、loongarch64、x86_64）和多种硬件平台（如 Raspberry Pi、QEMU 虚拟机等），采用模块化设计，注重兼容性、安全性与可移植性。

Undefined OS 提供了 POSIX 兼容的系统调用接口，致力于提供兼容 Linux 内核用户接口，同时通过层次化的系统调用设计，保留了未来兼容其他系统调用规范的接口。

Undefined OS 使用 Rust 语言编写，充分利用了 Rust 的安全性和性能优势，利用 Rust 提供的 RAII 机制和所有权系统，确保了资源获取和释放的安全性；利用锁和其他同步原语，确保内核在高并发场景下的稳定性和安全性。

Undefined OS 内核采用了模块化设计，核心内核模块包括硬件抽象层、进程管理、内存管理、文件系统、信号机制、网络支持等。每个模块都提供了清晰的接口和实现，便于扩展和维护。同时，部分内核组件不依赖于其他模块，可以作为独立的 crate，提供给其他 OS 实现或其他用户空间程序使用。

1.2 Undefined OS 内核整体设计

Undefined OS 是在 ArceOS 的基础上完成的，ArceOS 是一个基于 Rust 的 Unikernel 实现，以组件库的形式，为操作系统内核提供了基础设施和核心功能。Undefined OS 在 ArceOS 的基础上，进一步完善了文件系统、内存管理、进程管理等核心模块，将原本的 Unikernel 设计转变为一个完整的操作系统宏内核，在此基础上，我们实现了 200 个左右的 POSIX 系统调用，其中许多系统调用可以通过 LTP（Linux Test Project）测试套件进行验证，确保了 Linux 兼容性。

项目的总体架构延续了 ArceOS 的组件化的设计思想，采用模块化的方式，将内核功能划分为多个独立的模块，每个模块负责特定的功能。这样的设计使得内核更加灵活，易于扩展和维护，同时模块可以从内核中拆分出来，作为组件库提供给其他项目。Undefined OS 主要包括以下几个模块：

模块	功能
----	----

进程管理	支持线程和进程管理，支持进程组/会话管理
内存管理	负责用户和内核的内存分配，支持共享页面、大页等功能，提供了懒分配与 Copy-on-Write 等优化策略，支持 buddy, slab, tlf 等多种内存分配算法
硬件抽象层	提供了对不同硬件平台的抽象，支持多种体系结构和硬件平台，提供了统一的设备驱动接口，使用配置文件来描述结构和平台
任务调度	支持多种调度算法，包括 FIFO, Round Robin, 优先级/抢占调度等，支持实时任务调度，提供了可扩展的调度框架
文件系统	支持 Ext4, FAT32, RAMFS 文件系统，包括基本的文件操作和权限管理，提供了 procfs 和 sysfs 的部分功能支持，使用了可扩展的接口设计以支持新的文件系统
信号机制	支持进程/线程间信号机制，支持进程间的信号传递和处理，支持管道、共享内存等多种进程间通信方式
网络支持	支持 TCP/UDP 通信协议，提供了真实网络设备驱动接口，通过相关测试工具验证了网络模块的功能和性能
系统调用	将系统调用抽象为接口层与实现层，接口层提供了安全的类型检查与转换功能，实现层保证了系统调用的一致性和性能。两者结合保证了系统调用设计的可扩展性

目前上述模块的功能还在进一步完善中，我们希望通过不断的迭代和优化，提升系统的稳定性和性能，最终实现一个功能完善、性能优越的操作系统内核，同时也希望这些模块在功能不断完善后，可以被其他项目复用，以验证实现的通用性与可移植性。

1.3 实现参考与致谢

本项目的代码依赖于 ArceOS 组件库提供的部分功能，在开发的过程中我们对其代码进行了较多的修改。

此外，部分功能的实现参考了其他优秀开源项目的设计思路，例如 Linux 内核、星绽 OS 等。

部分代码实现参考了 StarryMix 内核的设计，在此向相关同学的帮助表示感谢。

第 2 章 系统概述

2.1 整体架构

内核的整体架构如下图所示：

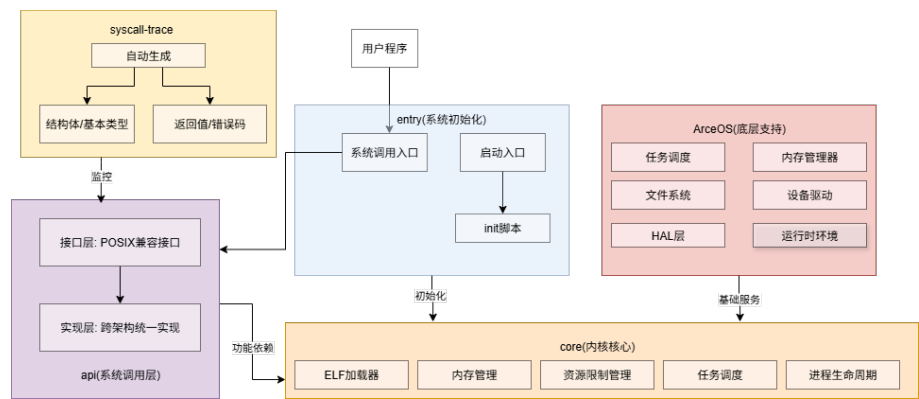


图 2-1 Undefined OS 内核架构图

Undefined OS 的总体架构如上图所示。

每一个模块将在接下来的章节里分别介绍。

2.2 代码结构

```
UndefinedOS
├── api                // 系统调用 API
│   ├── src
│   │   ├── core      // 系统调用核心功能
│   │   ├── imp        // 系统调用实现层
│   │   ├── interface  // 系统调用接口层
│   │   └── utils
│   └── arceos
│       ├── configs    // 硬件平台相关的配置文件
│       │   └── platforms
│       ├── modules
│       │   ├── axalloc // 内存分配模块
│       │   ├── axconfig // 配置模块
│       │   └── axdisplay // 显示模块
```

```

|   |   |─ axdma      // DMA模块
|   |   |─ axdriver   // 驱动模块
|   |   |─ axfs-ng    // 文件系统模块
|   |   |─ axhal      // 硬件抽象层
|   |   |─ axlog      // 日志模块
|   |   |─ axmm       // 内存管理模块
|   |   |─ axnet      // 网络模块
|   |   |─ axns       // PerCPU资源管理模块
|   |   |─ axruntime  // 运行时模块
|   |   |─ axsync     // 同步模块
|   |   |─ axtask     // 任务调度模块
|─ configs            // 架构特定的配置文件
|─ core               // 核心模块
|   └─ src
|─ modules            // 其他模块化的内核组件
|   └─ vfs
|─ process            // 进程管理模块
|─ scripts            // 编译脚本
|─ src                // 启动代码
└─ syscall_trace     // 系统调用追踪模块

```

2.3 组件化的设计理念

Undefined OS 内核延续了底层 Unikernel 设计的组件化理念，将内核功能划分为多个独立的模块，每个模块负责特定的功能。这样的设计使得内核更加灵活，易于扩展和维护，同时模块可以从内核中拆分出来，作为组件库提供给其他项目。此外，每个组件还可以通过 Rust 的 `feature` 机制进行功能的选择性编译，允许用户根据需要启用或禁用特定功能，或选用功能的不同实现，从而实现更高的灵活性和可定制性，适配不同的使用场景。

为了支持不同的架构和平台，Undefined OS 还采用了类似 Linux 设备树的配置文件机制。通过配置文件，我们可以描述硬件平台的特性和功能，内存地址空间布局，设备的 MMIO 地址等信息。这样，Undefined OS 可以在无需修改代码的前提下，轻松适配不同的硬件平台，编译时根据配置文件生成适合特定硬件平台的内核镜像。

例如，在支持硬件时钟的平台上，我们可以为 `axhal` 模块启用硬件时钟特征(通过 `rtc feature`)，同时在配置文件中设置 RTC 设备的 MMIO 地址，从而实现硬件时钟支持。而在不支持硬件时钟的平台上，则可以禁用该特征，相关的代码也不会被编译到内核中。

我们为许多功能提供了多个可选的实现，例如内存分配器支持 `Buddy`、`Slab`、`TLSF` 等多种算法，针对不同大小的内存（如 256MB，1GB 等）均提供了相关的特性，以便适配小内存的嵌入式设备和大内存的 PC 设备。

文件系统支持 `Ext4`、`FAT32` 等多种格式，可根据实际情况进行选择 and 裁剪。

任务调度支持 `FIFO`、`Round Robin`、优先级调度等多种算法。用户可以根据需要选择合适的实现，从而实现更高的灵活性和可定制性。例如，在实时性要求较高的系统上，我们可以选择优先级调度算法，以确保高优先级任务能够及时执行；而在资源受限的嵌入式系统上，则可以选择更简单的 `FIFO` 或 `Round Robin` 调度算法，以降低任务切换带来的开销。此外，我们还提供了可选的抢占式调度特性，允许在高优先级任务到来时中断低优先级任务的执行，从而提高系统的响应速度和实时性。

2.4 系统调用接口设计

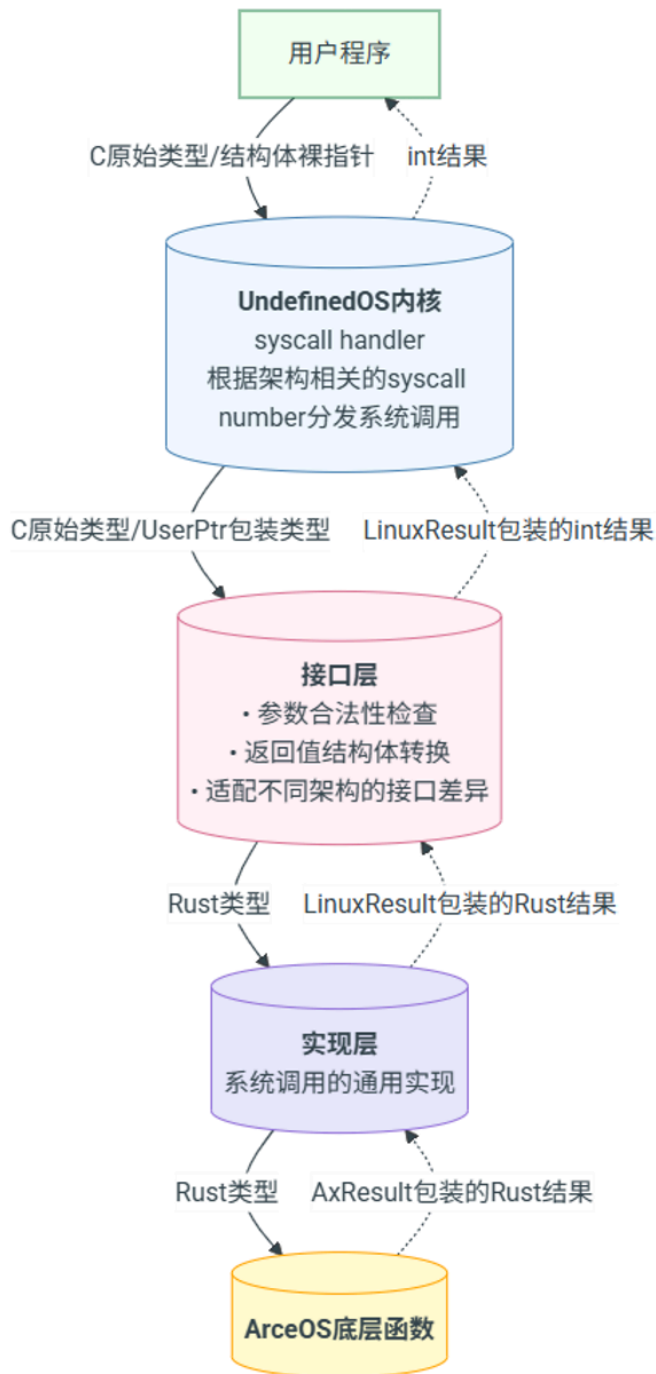
我们采取了接口和实现分离的系统调用设计。

这个设计的初衷是，我们发现许多系统调用有着相似的名字和类似的功能，往往存在部分系统调用的功能是重叠的，或者是包含在其他系统调用中的。这其中有一部分的历史原因，`Linux` 在演进的过程中，不可避免地会因为旧系统调用的缺陷而引入新的改进版本的系统调用，从而导致了系统调用的重复和冗余（例如 `stat`，`fstat`，`lstat`，`statx`，`fstatat`，以及 `poll`，`ppoll`，`select`，`pselect` 等系统调用）。

我们希望在实现系统调用时，能够将这些系统调用的实现进行整合，避免重复的代码和不必要的复杂性。因为如果为功能相近的若干系统调用分别实现一遍代码，可能在排查问题或者需要进行修复时，需要在多个地方进行修改，存在遗漏的风险。

此外，虽然我们的目标是提供 `Linux` 兼容的 `POSIX API`，但如果未来我们希望支持其他 OS 的 `API`，或者提供我们的自定义 `API`，那么这种结构的划分也能够让我们在实现时更加灵活。

具体设计结构如图所示：



具体地，我们整合一类功能相近或彼此包含的系统调用，实现一个通用的实现函数以完成它们的功能的并集，作为**实现**部分。然后为每个系统调用单独提供一个**接口**函数，作为对应用程序的接口，函数内进行参数的检查和转换，以合适的参数组合调用对应的实现函数。具体规范包含下面的部分：

- **实现**部分的函数名为 `sys_XXX_impl`，其中 `XXX` 为系统调用的名称。实现部分的函数参数和返回值都是 `rust` 原生类型，不涉及指针读写和数据结构转换等工作，只关心系统调用功能实现。
- **接口**部分的函数名为 `sys_XXX`，其中 `XXX` 为系统调用的名称。接口部分的函数参数和返回值为 `POSIX` 标准定义的类型，函数内进行参数的检查和转换，以合适的参数组合调用对应的实现函数。接口部分的函数需要处理指针读写和数据结构转换等工作，可能会涉及到 `unsafe` 代码。
- **接口**部分的函数参数主要有两类：一类是 `rust` 定义的 `c` 语言接口类型，如 `c_int` 等，另一类是指针类型。此处我们对用户指针类型进一步进行了语义化包装，命名为 `UserInPtr<T>`，`UserOutPtr<T>`，`UserInOutPtr<T>`，分别表示输入指针、输出指针和输入输出指针。我们在实现时，使用这些语义化的指针类型来替代原有的指针类型，以提高代码的可读性和可维护性，同时便于添加合适的权限和边界检查。
- **接口**部分的函数可能需要处理数据结构转换等工作，在**实现**部分返回的结构体为内核的内部类型，可能和 `POSIX` 标准中规定的和用户交互的类型有所不同。甚至，在不同的架构下，`Linux` 规定的数据结构也可能会有所不同，可能存在字段顺序不同、字段大小不同等问题。我们在实现时，使用 `#[repr(C)]` 属性来指定数据结构的内存布局，同时针对不同的架构，使用 `cfg` 宏来管理不同架构下的用户结构体。

第 3 章 开发板适配

3.1 RISC-V 昉·星光 2

首先，为了能够制作出可用于开发板的内核镜像，我们需要先修改编译时的配置与页表，并确保开发板上预装的 SBI 版本正确（否则内核运行成功也无法通过串口输出）。

编译出正确的二进制可执行文件后，使用下面的指令制作内核镜像：

```
mkimage -A riscv -O linux -C none -T kernel -a 0x40200000 -e  
1 0x40200000 -n "UNDEFINED OS on StarFive VisionFive2" -d undefined-  
os_riscv64-qemu-virt.bin kernel.uimage
```

代码 3-1

根据官方手册指引，我们首先需要使用电脑连接开发板上的串口，调整波特率等一系列参数，然后使用串口监视器与串口通信，以便进行后续的调试和测试。

接下来为开发板连接电源，插入已经烧录好内核镜像的 SD 卡，按下开发板上的复位按键，开发板会重新启动。

接下来使用下面两条指令，加载内核镜像到指定位置并执行，注意这里的地址需要必须和内核的.ld 链接脚本中定义的入口地址一致。

```
1 fatload mmc 1:3 0x40200000 kernel.uimage  
2 bootm 0x40200000 - ${fdtcontroladdr}
```

代码 3-2

3.1.1 执行流程

星光板 2 的启动流程为 SBI -> U-Boot -> Undefined OS 内核。

其中 OpenSBI 运行于 M 态；U-Boot 由 OpenSBI 引导，运行于 S 态；操作系统由 U-Boot 来加载，并运行于 S 态。

具体执行流程如下：

```

1  RISC-V硬件上电 (M态)
2  |
3  v
4  [OpenSBI] (M态)
5  |
6  |  初始化硬件
7  |  加载U-Boot到内存 (如0x8020_0000)
8  v
9  [U-Boot] (S态)
10 |
11 |  初始化外设 (网卡、SD卡等)
12 |  从SD卡/网络加载内核 (到0x4020_0000) 和设备树 (到0x4600_0000)
13 |  传递设备树地址 (a1寄存器) 和其他参数
14 v
15 [Undefined OS内核] (S态)
16 |
17 |  设置中断和异常处理
18 |  初始化MMU和页表
19 |  初始化设备驱动 (串口、时钟、SD卡等)
20 |  挂载文件系统
21 |  启动用户程序
22 v
23 用户空间 (应用程序)

```

代码 3-3

调试时的关键在于串口输出和内存地址的正确映射。

在 OpenSBI 和 U-Boot 阶段，串口已经初始化，可以通过串口查看启动日志。在 Undefined OS 内核中，早期启动阶段也可以使用串口输出调试信息（注意：在 MMU 启用前使用物理地址访问串口，启用后使用虚拟地址）。

同时，必须确保内核的链接地址（虚拟地址）与 U-Boot 加载的物理地址对应，且页表映射正确。

3.1.2 内存地址映射

当 U-Boot 加载引导 OS 镜像时，开始会解析并引导执行入口。

U-Boot 的执行入口地址非常重要，需要与内核的.ld 链接脚本中定义的入口地址一致。

因此在启动 os 时需要构建好内核页表，正确映射虚拟地址与物理地址的对应关系，正确设置内核的入口地址，是成功启动自开发 OS 的关键。

3.1.3 页表结构与设置

星光派 2 StarFive VisionFive2 开发板使用的 CPU 型号为 JH-7110，根据技术手册及 Linux 源码的 riscv 架构相关代码部分，我们重新构建了虚拟内存页表及页表项。

我们使用一个固定的偏移来转换物理地址与虚拟地址，这样便于内核访问，例如将 0xffff_ffc0_4000_0000 映射到物理地址 0x4000_0000，二者之间偏移量为 0xffff_ffc0_0000_0000。

3.1.4 关于固件

开发板中预装的固件(OpenSBI)版本较低，不支持目前最新的 RISC-V SBI 标准，因此我们最初无法在开发板上看到串口输出，直到更新了固件版本，问题才得以解决。

3.2 loongarch-龙芯 2k1000

截止本文档提交时（2025 年 8 月 17 日），我们还未完成对 龙芯 2k1000 开发板的支持，因此此处暂时不做介绍。

第 4 章 应用程序支持

在不断完善基础的系统功能和实现了大量的 Linux syscall 之后，我们认为我们的 OS 已经足够完善并且可以运行 Linux shell 和一系列实用程序了。

在经过一系列的调试和完善打磨后，我们测试了一系列的应用程序，经过验证它们可以在我们的 OS 上运行、编译，我们认为，这标志着我们的 OS 已经达到了实用级别。

4.1 发行版支持

为了验证我们 OS 的实用性，同时便于应用的移植，我们决定提供对 Ubuntu 根文件系统的支持。我们的 OS 目前可以运行 Ubuntu 24.04.2 LTS 根文件系统上的二进制程序，支持从 bash 登录至终端，可以通过 apt 安装软件包，可以运行通过 apt 安装的 git, gcc, g++ 等实用程序。

4.2 集成开发环境支持

我们计划支持文本编辑器（busybox vi/vim）、编译器（gcc、g++、rustc）和开发工具（git）以实现在自己的操作系统上进行实际开发大型项目。

当前进展：

- 已实现 busybox vi 的编辑功能支持
- GCC 和 G++ 可编译较为复杂的 C/C++ 程序
- Git 基础工作流已支持（详见下方说明）
- Rustc 支持仍在进行中

如果能够实现我们的目标，我们的操作系统将能够支持自举，能够在上面编译和运行自己的内核。

4.3 已支持应用程序列表

下面列出了部分已经支持的应用程序和工具：

4.3.1 BusyBox

BusyBox 是嵌入式系统的核心工具集，我们实现了其完整支持，这标志着操作系统具备了基础运维能力。已实现的关键常用功能主要包括：

- shell 环境 (ash)
- 文件操作工具 (ls, cp, mv, rm 等)
- 文本处理工具 (grep, sed, awk)

4.3.2 编译器支持 (Tcc、gcc、g++、rustc)

我们支持完善的多语言编译工具链，为系统开发环境提供坚实保障。

编译器	支持状态与验证场景
Tcc	完全支持,可编译 C 程序,可以在我们的操作系统上自举编译出一个 C 语言编译器
GCC	完全支持,可编译 C 程序
G++	完全支持,可编译 C++程序、STL 容器测试程序
Rustc	完整支持需要其他依赖，仍在开发中

4.3.3 git

我们实现了 Git 全工作流支持，包含大部分常用核心命令：

```
// 初始化与配置
git config --global core.pager ""
cd test
git init      // 创建裸仓库
git clone     // 克隆仓库

// 文件操作与提交
echo "123" > a.txt
git add .
git diff --staged
git status

// 用户身份设置
git config user.name dragon
git config user.email undefined@os

// 提交历史管理
git commit -m "add a"
git log
git show
```



```
// 分支操作
git checkout -b new_branch
mkdir foo
echo "new file" > foo/bar
git status

// 修改与合并
vi a.txt
git diff
git add . && git commit -m "new commit"
git branch -vv
git log
git checkout master
git merge new_branch
git log
```

总体而言，我们的操作系统已经能够支持一系列基本的 `git` 指令，并且在不断完善中。

4.3.4 BusyBox vi

`vi` 是一个轻量高效的文本编辑工具，支持基本的文本编辑功能，包括打开、编辑和保存文件。

我们的 OS 目前已经能够支持 `vi` 的完整编辑功能。

4.3.5 grep

我们已经支持 `grep` 工具，包括基础正则表达式搜索功能。例如：

支持在文件中搜索模式：`grep "pattern" file.txt` 支持递归搜索目录：`grep -r "pattern" /path` 支持显示行号：`grep -n "pattern" file.txt`

这些功能使得用户可以在我们的操作系统上进行文本搜索任务。

4.3.6 lua

我们支持 Lua 脚本解释器：包括完整 Lua 语法、标准库（`string`, `table`, `math` 等）、文件 I/O 基本操作、协程支持等。

4.3.7 python

我们支持 MicroPython，这是一个轻量级的 Python 实现，适用于嵌入式系统和资源受限的环境。在我们的操作系统上，MicroPython 可以运行基本的 Python 脚本。

目前支持的功能包括：

- 基础 Python 语法（如条件、循环、函数等）
- 部分标准库模块（如 `sys`, `math` 等）
- 交互式 REPL 环境

4.3.8 SQLite3

我们支持 SQLite3 数据库引擎，包括运行和编译。用户可以在我们的操作系统上使用 SQLite3 进行数据库操作。支持的功能主要包括：

- 创建数据库和表
- 执行 SQL 语句（如 `SELECT`, `INSERT`, `UPDATE`, `DELETE` 等）
- 事务支持

此外，SQLite3 在我们的操作系统上可以正确编译，这进一步证明了我们的操作系统的实用性和兼容性。

第 5 章 图形界面 (GUI) 支持

我们实现了一个简单的图形用户界面 (GUI) 系统, 支持基本的窗口管理、输入处理和图形渲染等。

系统采用 X11 显示协议作为基础, 移植轻量级的 Xfce 桌面环境作为用户界面。

在输入处理方面, 计划整合键盘、鼠标等输入设备驱动, 构建统一的输入事件处理系统, 特别关注终端驱动的实现, 使其能够正确处理如 Ctrl+C 等特殊按键组合。

在显示驱动方案上, 针对不同平台设计了两种实现路径: 在 QEMU 模拟环境中使用 virtio 帧缓冲驱动进行本地渲染; 在实际硬件平台上, 当本地显示驱动不可用时, 采用 X11 forwarding 技术将图形界面通过网络转发至其他设备的 X Server 进行远程渲染。

我们计划逐步迭代, 扩展系统功能, 最终实现完整的图形用户界面支持。

5.1 GUI 架构

整个 GUI 系统采用分层架构设计:

- 在硬件层直接管理输入设备 (键盘、鼠标) 和显示输出设备。
- 内核层提供标准设备接口, 包括实现 `/dev/input/events` 输入事件设备与 `/dev/fb0` 帧缓冲设备。
- 用户空间层则支持应用程序通过 Xlib 或 XCB 等图形库向 X Server 发送绘图请求, 最终由 X Server 通过内核的显示驱动管理模块执行实际的渲染操作。

这种分层设计确保了硬件访问的安全性和用户程序的灵活性。

在我们的设计中, 硬件层负责提供基本的图形输出能力, 内核层负责管理图形资源和提供接口, 用户程序层则实现具体的图形界面。

5.2 渲染支持

帧缓冲驱动 (一种直接操作屏幕显示内存的技术) 是我们的核心渲染支持组件, 目前已在 virtio-gpu 驱动 (QEMU 虚拟机的图形加速技术) 上取得进展。

系统提供了标准的帧缓冲设备接口 `/dev/fb0`, 使应用层可以直接访问显示缓冲区。在 QEMU 环境下, 我们已实现通过 virtio 驱动完成图形渲染的基本功能。

5.3 输入系统支持

输入子系统需整合多种输入源：物理键盘/鼠标设备、串口终端以及 SBI 控制台。在最初的版本中，我们在 RISC-V 平台上使用 SBI console，在 LoongArch 平台上使用串口，作为基本的输入输出源。

为了进一步支持交互型的控制台应用，例如 vim, bash 等，需要提供控制台管理相关的功能，例如获取终端的大小、处理信号相关的特殊按键等。我们需要实现完整的终端驱动，支持 cooked mode 处理机制，使得 shell 等命令行工具能够正常解释特殊按键组合，例如当用户按下 Ctrl+C 时产生 SIGINT 信号而非直接传输原始字符数据。

此外，为了进一步支持 x11 桌面环境，我们的目标是实现 `/dev/input/events` 等设备接口，使应用程序能够接收标准化的输入事件。

5.4 X11 协议支持

X11 是类 Unix 系统最常用的图形显示协议，它使用客户端-服务器架构。为了在 Undefined OS 中引入图形化支持，我们考虑率先支持有丰富的应用生态的 X11 协议。

X11 协议的著名实现是 X.Org Server，它提供了完整的图形显示和输入处理功能。我们计划通过以下步骤实现对 X11 的支持：

- 首先添加必要的驱动支持，包括显示和外设（键盘/鼠标等）支持。
- 实现 `/dev/fb0` 帧缓冲设备接口，允许应用程序直接访问显示缓冲区。
- 实现 `/dev/input/events` 设备接口，使应用程序能够接收标准化的输入事件。
- 尝试移植 Xorg Server，使其在我们的 OS 上运行。
- 尝试移植一些常用的 X11 应用程序，例如 xterm、gedit 等，以验证我们的 X11 支持。

5.5 Xfce 桌面支持

Xfce 是一个轻量级、快速且美观的桌面环境。在添加了对 X11 协议的支持后，我们准备尝试移植 Xfce 桌面环境。为了让 Xfce 在我们的系统上流畅运行，我们扩展了多项功能，主要包括：

- **close_range**：批量关闭文件描述符的技术，提高资源管理效率
- **epoll_pwait2**：增强型事件轮询系统，让界面能及时响应操作
- **IO 优先级控制**：智能分配磁盘读写资源

- **安全沙箱(seccomp):** 限制程序权限，防止恶意软件破坏系统

在进程管理方面，我们也添加了许多新功能以更好的支持 Xfce，包括：

- **CLONE_PIDFD** 参数支持：精确控制程序创建的进程
- **Futex 增强：**改进多线程同步机制
- **PIDFD 支持：**优化进程等待和监控功能

第 6 章 内存管理模块

Undefined OS 内核的内存管理模块负责实现虚拟内存管理、物理内存分配、地址空间管理、共享内存管理等功能。内存布局设计按照四种不同架构的规范进行不同的调整，确保内核与用户空间的有效隔离和高效访问。

6.1 内存布局

Undefined OS 中，内核与用户共享地址空间，可以在 `axconfig/platforms` 下动态配置操作系统的内存布局。

Undefined OS 的内存空间主要包括内核空间（其中包含内核代码、内核栈与内核堆）、用户空间（包括用户代码、静态数据、用户堆空间、用户栈空间）。

6.2 物理内存分配

Undefined OS 综合考虑了用户程序的整页分配需求和内核对象的分配需求，设计了高效的两级内存分配器。

Undefined OS 通过平台配置来获取平台可用的总内存的大小和范围，然后将物理内存页交给页级分配器管理。根据页级分配的特点，页级分配器使用 `bitmap` 来管理每个物理页的分配状态。在页级分配器下，我们设置了字节级的分配器，允许使用多种类型的分配算法，以适应不同的使用需求。

Undefined OS 内存分配的相关逻辑在 `axalloc` 模块中，其核心是其中定义的 `GlobalAllocator` 结构。

```
1 pub struct GlobalAllocator {
2     balloc: SpinNoIrq<DefaultByteAllocator>,
3     palloc: SpinNoIrq<BitmapPageAllocator<PAGE_SIZE>>,
4 }
```

代码 6-1

其中 `palloc` 是页级分配器，基于位图分配器管理未使用的页并进行页分配操作；而 `balloc` 是字节级分配器，提供了常用的以字节范围为单位的分配接口。

内核在每次申请内存时，首先尝试从字节级分配器进行分配，如果字节级分配器剩余可用内存不足，则会尝试从页级分配器中申请若干页物理内存放入字节级分配器中，然后再从字节级分配器中进行分配。

字节级的分配器支持多种分配算法，包括 Buddy、Slab、TLSF 等。用户可以根据实际需求选择合适的分配算法，以平衡内存分配的速度和碎片化程度。

页级的分配器使用 `bitmap` 来管理每个物理页的分配状态，支持按需分配和批量释放。

6.2.1 堆管理

Undefined OS 实现了基于 `brk` 的惰性堆管理机制：仅修改堆指针，物理页在首次访问时分配。

这样实现的优势是可以无锁更新堆边界，避免了多线程竞争，同时可以 $O(1)$ 时间复杂度调整堆大小，效率更高。

此外，Undefined OS 强制进行边界安全检查，有效的防止了堆溢出。

```
1 // brk系统调用实现
2 pub fn sys_brk(addr: usize) -> LinuxResult<isize> {
3     let process = current_process_data();
4     let heap_bottom = process.get_heap_bottom(); // 原子读取堆底
5
6     // 边界安全检查(不超过USER_HEAP_SIZE限制)
7     if addr != 0 && addr >= heap_bottom
8         && addr <= heap_bottom + USER_HEAP_SIZE
9     process.set_heap_top(addr); // 原子更新堆顶
10
11     Ok(process.get_heap_top() as _) // 返回当前堆顶
12 }
```

代码 6-2

6.3 虚拟内存管理

6.3.1 虚拟内存机制

Undefined OS 实现多种关键的虚拟内存安全机制和优化机制。

1. **写时复制(CoW)**: 在 `fork` 等场景下，不对内存页进行复制，仅复制页表项，同时设置页面为不可写的状态。在首次写入时触发物理页的复制，减少了内存拷贝开销。
2. **懒分配**: 在用户程序申请分配内存时，仅设置页表项并将其置为只读，在初次写入时才进行物理页分配。降低了启动延迟，同时减少了内存使用，可以有效适配稀疏内存或自己手动管理内存的应用程序

3. **用户指针检查:** 在访问系统调用中用户传递的指针前, 首先检查其权限, 以提供更好的安全性。同时, 考虑了写入用户内存时涉及 CoW 或懒分配时的正确处理。
4. **大页支持:** 为了提高大量内存的分配效率, 同时减少页表项的数量, Undefined OS 支持大页机制。通过使用 2MB 或 1GB 的大页, 系统能够显著降低 TLB 缺失率, 提高内存访问性能。Undefined OS 支持混合使用不同大小的页面, 以适应不同的内存需求和访问模式。
5. **共享内存支持:** Undefined OS 提供了高效的共享内存机制, 允许多个进程之间共享内存区域。通过使用共享内存, 进程可以避免不必要的内存复制, 提高数据访问效率, 同时提供了一种进程间通信的方式。共享内存区域的管理由内核负责, 确保不同进程之间的访问权限和数据一致性。Undefined OS 利用了 Rust 的所有权系统和引用计数机制来管理共享内存的生命周期, 确保在引用了同一块物理内存的所有进程退出后, 该物理内存可被正确回收。
6. **零拷贝文件映射支持:** 目前我们对文件映射支持还不够完善, 未来我们计划实现零拷贝文件映射机制, 允许文件直接映射到物理内存页, 减少数据复制, 提高文件访问性能, 降低内存消耗。

6.3.2 虚拟内存数据结构

Undefined OS 设计了一套高效的虚拟内存数据结构, 以支持复杂的内存管理需求。主要包括以下几个部分:

1. **页表:** 用于管理虚拟地址到物理地址的映射关系, 支持多级页表结构, 以降低内存占用和提高查找效率。
2. **反向映射表:** 用于记录物理页到虚拟页的映射关系, 支持快速查找和权限验证。
3. **交换区管理:** 用于管理被换出到磁盘的页, 支持按需加载和预读策略, 以提高内存使用效率。

6.3.3 页表管理

我们使用统一的抽象层来描述不同架构的页表, 提供了页面映射, 权限管理, 脏页标记, TLB 刷新等功能。

页表的实现依赖于具体的硬件架构, Undefined OS 提供了对 RISC-V、LoongArch、AArch64 和 x86_64 等架构的支持。统一的页表管理模块允许我们在不同架构下使用相同的接口进行虚拟内存操作, 同时根据具体架构的特点进行优化, 提高了可扩展性。

6.4 共享内存管理

在开发共享内存模块的过程中，我们面临了一系列关键设计抉择，这些决策直接塑造了当前系统的实现方式。核心理念在于平衡三个相互制约的目标：确保内存访问的高效性、维护系统资源的完整性、尊重用户程序的行为意图。这种平衡贯穿于整个开发历程，最终形成了现在的架构。

6.4.1 共享内存的生命周期管理

共享内存的生命周期管理是我们遇到的首要挑战。

在实现 `sys_shmctl` 系统调用时，我们采用立即删除策略：当用户通过 `IPC_RMID` 操作标记删除共享内存时，系统会立即将其从全局管理表（`SHARED_MEMORY_MANAGER`）中移除。

```
1  impl SharedMemoryManager {
2      pub fn delete(&self, key: u32) -> bool {
3          // 立即从B树映射表中删除键值对
4          self.mem_map.lock().remove(&key).is_some()
5      }
6  }
```

代码 6-3

选择立即删除而非标记删除，背后有两层关键考量：

1. 首先，这彻底杜绝了已被删除的共享内存段被重新附加的可能性。如果我们采用标记删除方案，就需要在每次 `sys_shmat` 调用时检查删除标志位，增加了系统调用的开销和复杂性。
2. 其次，这种设计与 Rust 的所有权系统完美契合——当共享内存对象从全局表中移除后，只要还有进程通过 `Arc` 引用持有该对象，物理内存就不会被释放；当最后一个引用消失时，`Drop` 实现会自动触发内存回收：

```
1  impl Drop for SharedMemory {
2      fn drop(&mut self) {
3          // 当所有Arc引用消失时自动执行物理页释放
4          global_allocator().dealloc_pages(self.addr,
5          self.page_count);
6      }
7  }
```

代码 6-4

这种基于引用计数的延迟释放机制既保证了即时的访问控制，又实现了安全的资源回收，解决了传统操作系统常见的“释放后使用”(use-after-free)漏洞风险。

6.4.2 进程级映射的优化设计

在进程数据结构中，我们采用了特定的映射关系设计：

```
1 pub struct ProcessData {  
2     // 虚拟地址到共享内存段的直接映射  
3     pub shared_memory: Mutex<BTreeMap<VirtAddr, Arc<SharedMemory>>>,  
4     // ...  
5 }
```

代码 6-5

这种设计直接服务于 `sys_shmdt` 系统调用的需求——当用户传递虚拟地址参数解绑共享内存时，系统能够以 $O(\log n)$ 时间复杂度快速定位对应的共享内存段。相比遍历所有共享内存段的 $O(n)$ 方案，这种基于 B 树的映射在进程附加大量共享内存段时（如数据库系统场景）能带来数量级的性能提升。

6.4.3 持久性与泄漏防范的平衡

共享内存是否应该在所有进程解绑后自动释放？

我们深入研究了 Linux 的行为规范：共享内存本质是持久性系统资源，生命周期独立于任何特定进程。因此**即使所有进程都已解绑，内存段仍会持续存在，直到显式删除或系统重启。**

这种设计选择的优势在于允许进程在临时解绑后重新附加同一内存段、支持跨会话协作（守护进程可以创建共享内存供后续会话使用）、避免误释放（可以防止因进程临时解绑导致关键数据丢失）。

当然，这也带来了潜在的内存泄漏风险。为此，我们支持与 Linux 兼容的管理工具链和相关的系统调用，如：

1. `ipcs` 命令：列出所有共享内存段及其附加计数
2. `ipcrm` 命令：通过 `sys_shmctl` 显式删除指定内存段

6.4.4 共享内存设计总结

总体而言，Undefined OS 的实现复刻了 Linux 的核心特性，主要包括：

1. 共享内存与进程解耦，生命周期独立
2. 显式删除机制：需主动调用 `IPC_RMID`

3. 引用计数控制：通过 Arc 实现零拷贝内存回收

4. 运维友好：兼容标准 SysV IPC 管理工具

同时，Undefined OS 通过 Rust 所有权机制天然规避了 use-after-free 风险，同时保持与 POSIX 标准的严格兼容性。

6.5 优化设计

Undefined OS 实现了写时复制（Copy-On-Write, COW）和 懒分配（Lazy Allocation）两种机制，用于优化内存管理性能。

这两种机制均能有效降低内存开销和启动延迟。CoW 特别适用于 fork-exec 模式（如 shell 启动新进程），而懒分配则优化了堆内存使用。

在实际测试中，这两种机制显著减少了进程创建时间和内存占用。

6.5.1 写时复制(CoW)

写时复制是一种优化内存使用的技术，主要用于共享内存的场景（如 fork 创建子进程时）。

当父进程创建子进程时，父子进程共享相同的物理内存页，这些页被标记为只读。当任何一个进程尝试写入这些共享页面时，会触发缺页异常。此时，内核会为写入进程复制该页，并修改页表映射，使得父子进程各自拥有独立的物理页副本。

这样，只有实际发生写入的页面才会被复制，从而节省了内存并提高了效率。

具体的实现细节包括：

1. 共享映射:在 fork 时，子进程复制父进程的页表，并将所有可写页面标记为只读（同时记录为 CoW 页）。

```

1 pub fn try_clone(&mut self) -> AxResult<Self> {
2     for area in self.areas.iter() {
3         // 遍历当前地址空间的所有内存区域,克隆内存区域的后端处理程序
4         let backend = match area.backend() {
5             // 处理分配类型的内存区域 (带COW特性)
6             #[cfg(feature = "cow")]
7             Backend::Alloc { populate: _, align } => {
8                 // 为了防止后续映射立即建立页表项
9                 Backend::new_alloc(false, *align)
10            }
11            // 对于其他类型的后端,直接克隆
12            other => other.clone(),
13        };
14
15        // 在启用COW特性时,创建只读权限标志
16        #[cfg(feature = "cow")]
17        let cow_flags = area.flags() - MappingFlags::WRITE;
18
19        // 遍历内存区域中的每个页面
20        for vaddr in PageIterWrapper::new(area.start(), area.end(),
21            align)
22        {
23            .expect("Failed to create page iterator")
24            {
25                // 尝试查询当前地址空间的页表
26                match self.pt.query(vaddr) {
27                    Ok((paddr, _, page_size)) => {
28                        // 启用COW特性时的处理逻辑
29                        #[cfg(feature = "cow")]
30                        {
31                            // 将原始地址空间的页表项改为只读
32                            self.pt
33                                .protect(vaddr, cow_flags) // 更新页表
34                                // 项权限
35                                .map(|(_, tlb)| tlb.flush()) // 刷新TLB
36                                .expect("protect failed");
37
38                            // 在新地址空间映射相同的物理页面 (但设为只读)
39                            new_aspace
40                                .pt
41                                .map(vaddr, paddr, page_size,
42                                    cow_flags) // 映射到相同物理地址
43                                .map(|tlb| tlb.flush()) // 刷新TLB
44                                .expect("map failed");
45                        }
46                    }
47                }
48            }
49        }
50    }
51 }

```

代码 6-6

2. 异常处理：当发生写入导致的缺页异常时，在启用 cow 特性的情况下，内核将检查该页是否为 CoW 页。如果是，则分配一个新的物理页，复制原页内容，并重新映射到故障地址（标记为可写），然后恢复进程执行。（相关处理逻辑在内核的页 fault 处理函数中，如下所示）

```

1  pub fn handle_page_fault(&mut self, vaddr: VirtAddr, access_flags:
2  MappingFlags) -> bool {
3      // 尝试在地址空间中查找包含该虚拟地址的虚拟内存区域（VMA）
4      if let Some(area) = self.areas.find(vaddr) {
5          // 获取该内存区域的原始标志（权限）
6          let orig_flags = area.flags();
7
8          // 检查访问权限是否在原始标志的权限范围内（即访问是否合法）
9          if orig_flags.contains(access_flags) {
10             // 进入此分支有两种情况：
11             // 1. 共享页面（如果VMA中有共享页面）
12             // 2. 写时复制（Copy-on-Write, COW）
13
14             // 如果启用了写时复制（COW）特性，并且当前访问是写操作
15             #[cfg(feature = "cow")]
16             {
17                 // 尝试查询页表，获取该虚拟地址对应的物理地址等信息
18                 if access_flags.contains(MappingFlags::WRITE)
19                     && let Ok((paddr, _, page_size)) =
20 self.pt.query(vaddr)
21                 {
22                     // 处理写时复制（COW）缺页：
23                     // 1. 由写操作引起的页错误
24                     // 2. 页表项（PTE）存在（说明之前已经映射过）
25                     // 3. 不是共享内存（因为共享内存不需要COW）
26                     return Self::handle_cow_fault(
27                         vaddr,          // 发生错误的虚拟地址
28                         paddr,          // 对应的物理地址
29                         orig_flags,    // 原始标志
30                         page_size,     // 页面大小
31                         &mut self.pt, // 页表引用
32                     );
33                 }
34             }
35             // ...如果不是COW情况，或者没有启用COW特性，则调用后端内存
36             // 区域的处理函数处理页错误
37         }
38     }
39 }

```

代码 6-7

3. 维护引用计数：每个物理页维护一个引用计数，当多个进程共享时计数增加；当进程复制自己的副本后，原页的引用计数减少。当计数归零时，物理页被释放。

6.5.2 懒分配(Lazy Allocation)

懒分配是一种延迟分配物理内存的策略。

当用户程序通过系统调用（如 `sbrk`）请求更多内存时，内核仅扩展进程的虚拟地址空间（VMA），并不立即分配物理页。当程序首次访问这些新区域时，会触发缺页异常，内核此时才分配物理页并建立映射。这避免了不必要的内存分配（例如，程序可能分配大量内存但未完全使用）。

具体的实现细节包括：

1. 虚拟地址扩展：在 `sys_brk` 系统调用中，内核只更新进程的堆结束地址（`brk`），而不分配物理页。

```

1 pub fn sys_brk(addr: usize) -> LinuxResult<isize> {
2     // 获取当前进程的堆顶地址作为默认返回值
3     let mut return_val: isize =
4     current_process_data().get_heap_top() as isize;
5     let heap_bottom = current_process_data().get_heap_bottom();
6
7     // 检查请求地址是否在合法范围内
8     if addr != 0 && addr >= heap_bottom && addr <= heap_bottom +
9     axconfig::plat::USER_HEAP_SIZE {
10         // 懒分配核心操作：仅更新堆顶指针，不分配物理页
11         current_process_data().set_heap_top(addr);
12         return_val = addr as isize;
13     }
14     // 返回更新后的堆顶地址
15     Ok(return_val)
16 }
```

代码 6-8

2. 缺页处理：当进程访问尚未映射的虚拟地址时，内核检查该地址是否在合法的 VMA 范围内。如果是，则分配物理页，建立页表映射，然后恢复进程；否则发送 `SIGSEGV` 信号终止进程。（相关处理逻辑在内核的页 `fault` 处理函数中，如下所示）

```
1  pub fn handle_page_fault(&mut self, vaddr: VirtAddr, access_flags:
MappingFlags) -> bool {
2      // 检查发生页错误的虚拟地址是否在当前地址空间的有效范围内
3      if !self.va_range.contains(vaddr) {
4          return false; // 地址无效, 无法处理
5      }
6
7      // 尝试在地址空间中查找包含该虚拟地址的内存区域
8      if let Some(area) = self.areas.find(vaddr) {
9          // 获取该内存区域的原始访问权限标志
10         let orig_flags = area.flags();
11
12         // 检查请求的访问权限是否在原始权限范围内
13         if orig_flags.contains(access_flags) {
14             // ...处理写时复制(CoW)相关逻辑
15
16             // 懒分配核心操作: 调用后端处理程序分配物理页
17             return area
18                 .backend() // 获取内存区域的后端处理程序
19                 .handle_page_fault(vaddr, orig_flags, &mut
self.pt);
20         }
21     }
22
23     // 如果找不到对应的内存区域或权限检查失败, 返回false
24     false
25 }
```

代码 6-9

上述懒分配机制通过延迟物理内存分配, 有效提高了 Undefined OS 的内存利用率。

第 7 章 进程管理模块

我们实现了一个独立模块 `process`，它是操作系统无关的，可以作为一个基础组件提供给其他操作系统使用，没有依赖 ArceOS 或者 Undefined OS 中的组件。

该模块采用分层架构，严格遵循 Linux 的进程组织结构：会话(Session)包含进程组(Process Group)，进程组包含进程(Process)，进程包含线程(Thread)。同时维护进程间的父子关系树，为操作系统提供完整的进程生命周期管理能力。

在我们的设计中，我们认为调度的基本单元是线程，我们据此设计了操作系统无关的线程和进程模型，旨在提供一个通用的包，来提供进程和线程管理操作，基本覆盖了 POSIX 的所有进程/线程操作，如：

- 会话/进程组/进程/线程创建
- 线程/进程退出(包括 `exit_group`)和收养
- 更改进程所在的进程组
- `pid/tid` 查询和管理
- 进程 `fork` 和创建新线程

以下是具体的细节说明。

7.1 核心数据结构

7.1.1 线程(Thread)

```
1 pub struct Thread {  
2     tid: Pid,           // 线程ID  
3     process: Weak<Process>, // 所属进程的弱引用  
4 }
```

代码 7-1

线程是最基本的执行单元，共享进程的内存空间和资源（在 linux 中，通过 `clone()` 系统调用的标志来控制资源共享），但有独立的栈和寄存器状态。

每个线程有唯一的 TID（线程 ID）。通过弱引用关联所属进程，避免循环引用。

在 Undefined OS 中，我们通过对每个进程都维护一张线程表来实现对线程的管理。

7.1.2 进程 (Process)

```
1 pub struct Process {
2     pid: Pid,                // 进程ID
3     threads: Mutex<BTreeMap<Pid, Arc<Thread>>>, // 线程表
4     process_group: Mutex<Weak<ProcessGroup>>, // 所属进程组
5     children: Mutex<BTreeMap<Pid, Arc<Process>>>, // 子进程
6     parent: Mutex<Weak<Process>>, // 父进程
7     is_zombie: AtomicBool,    // 僵尸状态标记
8     exit_code: AtomicI32,    // 退出状态码
9 }
```

代码 7-2

进程是程序执行的实例，包含独立的内存空间、资源和执行状态。

每个进程有唯一的 PID（进程 ID），是资源分配的基本单位。

Undefined OS 通过进程组实现进程分组，每个进程组的 id 也就是组长的 process id。

根据创建时的父子关系形成进程树，以 init 进程(PID=1)作为根节点

每个进程的执行状态可能是运行、就绪、阻塞、僵尸等，操作系统根据不同的运行状态会进行不同的处理。具体来说：

1. **运行状态（Running）**：进程正在 CPU 上执行指令。此状态下，操作系统通过调度器分配 CPU 时间片，确保进程高效运行。

处理逻辑：

- 在 `sys_execve_impl` 中，当进程加载新程序时，需要重置地址空间、设置入口点（`tf.set_ip()`），并更新用户栈（`tf.set_sp()`），使进程进入运行状态。

```

1  pub fn sys_execve_impl(
2      tf: &mut TrapFrame,
3      path: String,
4      args: Vec<String>,
5      envs: Vec<String>,
6  ) -> LinuxResult<isize> {
7      // .....
8
9      let process_data = current_process_data();
10
11     let addr_space = &process_data.addr_space;
12     let mut addr_space = addr_space.lock();
13     addr_space.unmap_user_areas()?;
14
15     // .....
16
17     // reset some process attributes
18     *process_data.signal.actions.lock() = Default::default();
19     process_data.shared_memory.lock().clear();
20
21     // new user context
22     tf.set_ip(entry_point.as_usize()); //设置入口点
23     tf.set_sp(user_stack_base.as_usize()); //更新用户栈
24
25     Ok(0)
26 }

```

代码 7-3

- `sys_clone_impl` 创建新进程时, 通过 `create_user_task` 初始化任务上下文, 并调用 `axtask::spawn_task` 启动调度, 使新进程直接进入运行队列。
 - 信号处理 (如 `sys_rt_sigreturn`) 在中断后恢复执行时, 通过 `tf.retval()` 恢复寄存器状态, 确保进程继续运行。
2. **就绪状态 (Ready)**: 进程已准备好运行, 但因 CPU 资源不足而等待调度。操作系统将其放入就绪队列, 待调度器分配 CPU。

处理逻辑:

- `sys_sched_yield` 主动让出 CPU, 使当前进程从运行态转为就绪态, 允许其他进程运行。
- 在 `sys_futex` 的 `FUTEX_WAKE` 操作中, 当等待队列被唤醒 (`wq.notify_one()`), 阻塞的进程转为就绪态, Undefined OS 通过 `axtask::yield_now()` 触发调度器重新分配 CPU。

- 信号递送（如 `send_signal_thread`）时，若目标进程未阻塞，信号处理程序将其状态标记为就绪，等待调度。
- 3. **阻塞状态 (Blocked)**：进程因等待事件（如 I/O、信号或资源）而暂停执行。操作系统将其移出调度队列，直到事件完成。

处理逻辑：

- `sys_futex` 的 `FUTEX_WAIT` 操作中，进程检查地址值后调用 `wq.wait()` 或 `wq.wait_timeout()` 进入阻塞态，随后将其加入等待队列（`futex_table`），直到其他线程通过 `FUTEX_WAKE` 唤醒。
- `sys_nanosleep` 通过 `api::sys_nanosleep` 实现睡眠，进程在指定时间内阻塞，使用计时器管理唤醒。
- 信号等待（如 `sys_rt_sigtimedwait`）时，进程调用 `current_thread_data().signal.wait_timeout()` 阻塞，直到信号到达或超时。
- 4. **僵尸状态 (Zombie)**：进程已结束（调用 `exit`），但其父进程尚未通过 `wait` 回收资源，此时保留退出状态供父进程读取。

处理逻辑：

- `sys_exit_impl` 处理退出逻辑：进程设置退出代码后，通过 `current_thread().exit()` 标记为僵尸并发送 `SIGCHLD` 信号通知父进程，并唤醒等待队列。
- 父进程在 `sys_wait4` 中检查子进程状态：通过 `child.is_zombie()` 判断僵尸进程，收集退出代码，并释放资源。若未设置 `WNOEXIT` 标志，则立即回收僵尸进程。
- 多线程场景中，若进程有未退出的线程，将会强制发送 `SIGKILL`（`send_signal_thread`）终止所有线程，确保僵尸状态正确清理。

状态转换与资源管理

通过系统调用（如 `exit`、`wait`、`clone`）或外部事件（如信号、资源可用）均可驱动状态切换。

在状态变更时需要同步资源，例如：

- 运行 → 阻塞：保存寄存器状态，释放 CPU。
- 僵尸 → 回收：清理地址空间和文件描述符。

7.1.3 进程组(ProcessGroup)

```

1 pub struct ProcessGroup {
2     pgid: Pid,                // 进程组ID
3     processes: Mutex<BTreeMap<Pid, Arc<Process>>>, // 组内进程
4     session: Weak<Session>,   // 所属会话的弱引用
5 }

```

代码 7-4

进程组是相关进程的集合(如 Shell 管道),共享相同的进程组 ID(process group ID, 等于组长的 PID)。

进程组是操作系统管理进程的重要机制,它将相关进程组织在一起,这种设计在 Shell 管道操作中尤为常见,例如 `ls | grep txt` 命令会将 `ls` 和 `grep` 两个进程置于同一个进程组中。

进程组便于管理组内所有进程,例如向 PGID 发信号会通知整个组。

在 Undefined OS 中,空的进程组会自动销毁。

```

1 // ProcessGroup::remove_process
2 impl ProcessGroup {
3     /// - `pid`: 要移除的进程的ID
4     pub fn remove_process(&self, pid: Pid) {
5         // 从进程映射表中移除指定PID
6         self.processes.lock().remove(&pid);
7
8         // 检查进程组是否为空
9         if self.processes.lock().is_empty() {
10             // 尝试升级会话的弱引用为强引用
11             if let Some(session_ref) = self.session.upgrade() {
12                 // 从会话中移除当前进程组
13                 session_ref.remove_process_group(self.pgid);
14             }
15
16             // 从全局进程组表中移除当前组
17             PROCESS_GROUP_TABLE.lock().remove(&self.pgid);
18         }
19     }
20 }

```

代码 7-5

7.1.4 会话(Session)

```

1 pub struct Session {
2     sid: Pid,    // 会话ID
3     process_groups: Mutex<BTreeMap<Pid, Arc<ProcessGroup>>>, // 进程
4     组
5 }

```

代码 7-6

会话是进程组的集合，是操作系统中的高级进程组织单元，代表用户的一次完整登录会话，有唯一的会话 ID（session ID），会话 ID 等于创建会话的进程组 ID。

一个会话包含多个进程组，并与控制终端（Controlling Terminal）关联，负责管理用户与系统的交互。

在 Undefined OS 中，空的会话会自动销毁。

```

1 // Session::remove_process_group
2 impl Session {
3     /// - `pgid`：要移除的进程组ID
4     pub fn remove_process_group(&self, pgid: Pid) {
5         // 从进程组映射表中移除指定PGID
6         self.process_groups.lock().remove(&pgid);
7
8         // 检查会话是否为空
9         if self.process_groups.lock().is_empty() {
10             // 从全局会话表中移除当前会话
11             SESSION_TABLE.lock().remove(&self.sid);
12         }
13     }
14 }

```

代码 7-7

7.2 PID/TID 管理

在操作系统内核中，进程标识符 (PID) 与线程标识符 (TID) 的分配是基础且关键的环节。考虑到按照 linux 规范，进程 ID (PID) 在逻辑上等同于其主线程的线程 ID (TID)，Undefined OS 采用了一个全局共享的原子计数器来统一且保证所有 PID 和 TID 的全局唯一性。

```

1 static NEXT_PID: AtomicU32 = AtomicU32::new(1);
2 // 静态全局原子计数器，用于生成下一个唯一的PID或TID。初始值为1。
3 fn generate_next_pid() -> Pid {
4     NEXT_PID.fetch_add(1, Ordering::Acquire)
5 }

```

代码 7-8

这里使用了原子操作的 `fetch_add` 方法将计数器递增 1，并返回旧值，同时使用 `Ordering::Acquire` 保证此获取操作的内存顺序正确性。

7.3 层次结构

Undefined OS 严格遵循 Linux 的规范实现了会话(Session)、进程组(Process Group)、进程(Process)、线程(Thread)之间的包含关系。

1. 每个会话拥有一名会话组长 (session leader)，通常是创建该会话的进程组组长。
2. 每个进程组拥有一名组长 (leader)，通常是最先创建该组的进程。
3. 每个进程拥有一个主线程 (main thread)。

ID 分配也需要遵循一定规则：

1. 进程 ID (PID) 在数值上等于该进程的主线程 ID (TID)。
2. 进程组 ID (PGID) 在数值上等于该组组长进程的 PID。
3. 会话 ID (SID) 在数值上等于该会话组长所在进程组的 PGID。

这种统一分配、层级复用的设计确保了：

1. 全局唯一性保证：所有 PID 和 TID 通过同一个原子计数器分配，绝对避免冲突。
2. 清晰的层级关系：通过 PID 复用 TID、PGID 复用 PID、SID 复用 PGID 的方式，明确体现了进程、线程、进程组和会话之间的隶属关系。
3. 资源自动回收：当进程、线程、进程组或会话等对象被销毁时，其所分配的 PID/TID/PGID/SID 自动失效，后续可通过计数器分配新的唯一 ID。

7.4 进程创建与 fork 实现

创建一个全新的进程涉及以下核心步骤：

1. 分配唯一标识符：为即将创建的新进程分配一个全局唯一的 PID。
2. 初始化进程结构：创建核心数据结构 `Process`，设置其初始状态（如状态为 `TaskStatus::Ready`）、内存空间框架（如页表初始内核映射）等。
3. 纳入进程组：将新进程添加到指定的目标进程组中，建立组成员关系。
4. 建立父子关系：明确设置新进程的父进程，并在父进程的 `children` 列表中记录该新进程。
5. 创建主线程：为该进程创建其第一个也是最重要的线程——主线程。主线程的 TID 将等于进程的 PID。

6. 注册到全局结构：将新创建的 Process 对象注册到系统全局的进程管理表中，以便后续查询和管理。

fork 系统调用实现：

fork 操作是类 Unix 系统中创建新进程的核心机制。在 Undefined OS 中，fork 的行为严格遵循 Linux 规范：

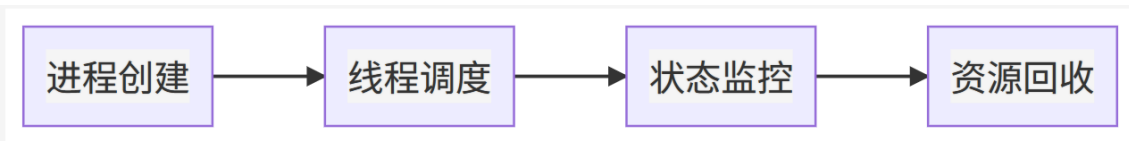
1. 行为描述：fork 创建当前进程（父进程）的一份几乎完全相同的副本，称为子进程。
2. 父子关系：新创建的子进程以调用 fork 的进程（父进程）作为其父进程。操作系统内核负责维护进程间的父子关系树形结构。
3. 继承关系：子进程继承父进程所属的进程组 (Process Group) 和会话 (Session)，即子进程与父进程位于同一个进程组和会话中。
4. 关键差异：fork 后父、子进程获得不同的返回值。父进程获得子进程的 PID，子进程获得 0。这是父、子进程在代码执行路径上产生差异的关键点。

```

1  // 作为进程对象(Process)的方法实现, self 代表调用 fork 的当前进程 (父进
   程)
2  pub fn fork(self: &Arc<Self>) -> Arc<Process> {
3      // 核心步骤: 生成一个全局唯一的 PID 给将要创建的子进程
4      let pid = generate_next_pid();
5
6      // 关键调用: 创建新进程
7      //   - 传入: 新分配的 PID
8      //   - 父进程: 传入当前进程对象的 Weak 引用 (&Arc<Self> 通过
   Arc::downgrade 转换)
9      //   - 进程组: 继承自当前进程 (self), 取其所属的进程组对象并转换为
   Weak 引用
10     let new_process = create_process(pid, Arc::downgrade(self),
   Arc::downgrade(&self.get_group()));
11
12     // 返回新创建的子进程对象的强引用 (Arc)
13     new_process
14 }
```

代码 7-9

7.5 生命周期管理



7.5.1 线程创建

Undefined OS 采用统一线程模型，所有执行单元（包括主线程）均通过标准流程创建。其核心设计包括：

1. **全局唯一 ID 池：** 线程 ID(TID)与进程 ID(PID)共享同一个全局计数器 (NEXT_PID)
2. **双级注册机制：** 新建线程会同时注册到其所属进程内部的线程列表（由进程对象管理）、注册到内核全局的线程管理表 (THREAD_TABLE)，供系统范围查询。
3. **资源继承模型：** 新创建的线程继承并共享其所属进程的资源命名空间，包括地址空间（内存页表、堆、栈区域）、文件描述符表、信号处理器表、用户 ID/组 ID 等。

主线程的特殊性设计：

作为进程的核心执行单元，主线程具有区别于普通线程的关键特性：

特性	主线程	普通线程
ID 关联	TID = PID(强制等价)	TID $\neq$ PID
exec 操作权限	唯一可执行 <code>execve</code> 替换进程镜像	无权执行
生命周期绑定	退出即终止整个进程	退出不影响进程存活
资源回收触发	退出时触发进程级资源回收	仅释放线程私有资源
父进程通知	退出时向父进程发送 SIGCHLD	无通知机制

线程创建的关键工作流程为：

[调用 `create_thread`] -> [分配全局唯一 TID]-> [创建线程控制块]-> [进程级注册]-> [系统级注册]-> [继承进程资源]

进程终止流程为：

[主线程退出] -> [终止所有附属线程]-> [释放进程地址空间]-> [关闭所有文件描述符]-> [解除共享内存映射]-> [发送 SIGCHLD 给父进程]

这样的设计优势主要在于：

1. 强一致性保障：
 - TID/PID 统一分配避免 ID 冲突
 - 双注册机制确保线程状态全局可见

2. 资源管理高效性:

- 资源继承减少初始化开销
- 层级回收防止资源泄漏

3. POSIX 兼容性:

- 主线程特权符合 POSIX 标准
- `execve` 行为保持与传统 UNIX 一致

4. 安全隔离:

- 普通线程崩溃不扩散至进程
- 主线程作为唯一可信执行环境

总的来说, Undefined OS 通过精密的 ID 管理和资源继承模型, 在保障 POSIX 兼容性的同时, 实现了高效可靠的线程生命周期管理。

7.5.2 线程退出

线程退出是操作系统资源回收的重要环节。

线程退出是操作系统资源回收的重要环节, 通过以下代码实现:

```
1  /// 线程退出函数
2  ///
3  /// # 参数
4  /// - `exit_code`: 线程退出状态码
5  pub fn exit(self: &Arc<Self>, exit_code: i32) {
6      // 1. 通知所属进程移除当前线程, 并传递退出状态码
7      //    - 内部处理线程资源的清理工作
8      self.get_process().remove_thread(self.tid, exit_code);
9
10     // 2. 锁定全局线程表并移除当前线程条目
11     //    - 确保线程ID不再被其他操作访问
12     THREAD_TABLE.lock().remove(&self.tid);
13
14     // 3. 线程对象离开作用域后自动释放资源
15     //    - Arc引用计数归零时触发Drop实现
16 }
```

代码 7-10

线程退出流程为:

1. 从所属进程的线程表中移除自身
2. 更新进程的线程状态信息
3. 从全局线程表中注销

4. 如果是进程的最后一个线程，触发进程退出

7.5.3 进程退出

进程退出涉及更复杂的资源回收和状态转换：

```

1  /// 进程退出函数（内部实现）
2  ///
3  /// # 关键处理步骤
4  pub(crate) fn exit(self: &Arc<Self>) {
5      // 1. 设置僵尸状态标志
6      //     - 保留进程退出码等关键信息
7      self.set_zombie();
8
9      // 2. 子进程重新托管
10     //     - 查找合适的收养者（通常是init进程）
11     if let Some(reaper) = self.get_child_reaper() {
12         // 遍历所有子进程并修改其父进程指针
13         for child in self.children.lock().values() {
14             child.set_parent(reaper.clone());
15         }
16     }
17
18     // 3. 进程组关系处理
19     //     - 如果当前进程是进程组组长，需要特别处理
20     if self.is_group_leader() {
21         // 从进程组中移除并检查进程组状态
22         self.get_group().remove_process(self.pid);
23     }
24
25     // 4. 父进程关系解除
26     //     - 从父进程的子进程列表中移除
27     if let Some(parent) = self.get_parent() {
28         parent.children.lock().remove(&self.pid);
29     }
30
31     // 5. 资源清理（文件描述符、内存映射等）
32     //     - 在父进程调用wait后由内核完成
33 }

```

代码 7-11

进程退出的关键步骤包括：

1. 状态标记为僵尸状态，保留退出信息供父进程查询
2. 处理子进程的托管关系
3. 从进程组中移除自身
4. 从父进程的子进程列表中移除
5. 资源回收（在父进程 wait 后完成）

7.5.4 进程组创建/迁移

进程组是操作系统管理进程的重要组织单元，允许对相关进程进行批量操作。

创建新进程组的核心逻辑如下：

1. 新组 ID 等于创建者 PID：确保组标识符的全局唯一性。
2. 创建者从原来的进程组迁移出来，自动成为组长：这与 linux 一致，简化权限管理。
3. 新进程组加入原进程组所属会话，从而保持会话结构完整性。

进程组迁移是动态调整进程组织结构的核心操作，遵循严格的约束条件：

1. 会话组长保持固定位置，任何情况均不可迁移。
2. 进程组长保持固定位置,不可迁移，只能先将组长移交给组内其他进程再进行迁移。
3. 目标进程组必须与原进程组属于同一会话，严禁跨会话迁移，保持会话边界完整性。

```

1  /// 迁移当前进程到指定进程组
2  pub fn move_to_group(self: &Arc<Self>, new_group: Pid) -> bool {
3      // 1. 会话组长保护机制
4      //     - 会话组长保持固定位置
5      if self.is_session_leader() {
6          return false;
7      }
8
9      // 2. 获取当前会话上下文
10     let session = self.get_session();
11
12     // 3. 查找目标进程组
13     //     - 目标组必须属于同一会话
14     let new_group = session.get_process_group(new_group);
15
16     // 4. 执行迁移操作
17     if let Some(target_group) = new_group {
18         // 更新进程的进程组指针
19         self.change_group(&target_group);
20         true // 迁移成功
21     } else {
22         false // 目标组不存在
23     }
24 }
```

代码 7-12

7.6 数据分离设计

Undefined OS 采用元数据与运行时数据分离的设计模式，实现操作系统核心模块与具体实现的解耦：

```
1  /// 进程控制块（操作系统无关）
2  pub struct Process {
3      pid: Pid,                // 进程ID
4      parent: Weak<Process>,   // 父进程引用
5      children: Mutex<BTreeMap...>, // 子进程列表
6      group: Weak<ProcessGroup>, // 所属进程组
7      ...
8  }
```

代码 7-13

```
1  /// 进程运行时数据（操作系统相关）
2  pub struct ProcessData {
3      pub addr_space: Arc<Mutex<AddrSpace>>, // 虚拟地址空间管理
4      pub resource_limits: Arc<Mutex<ResourceLimits>>, // 资源限制策略
5      pub signal: Arc<ProcessSignalManager>, // 信号处理机制
6      ...
7  }
```

代码 7-14

其中，Process 结构体的核心功能为管理进程身份信息（PID、父子关系）、维护组织结构（进程组、会话）、提供通用接口。

而 ProcessData 结构体的核心功能为管理硬件相关资源、实现平台特定行为等。

这种元数据与运行时数据分离的设计模式的优势主要包括：

1. 进程管理模块可独立发布为 crate
2. OS 具体实现通过 ProcessData 扩展
3. 避免核心模块依赖特定 OS 功能

7.7 资源限制

Undefined OS 支持 16 种 POSIX 资源类型，按 POSIX 标准实现了软/硬限制双机制：用户进程可自由调整 soft 值（不超过 hard），仅特权进程可修改 hard 值，防止资源滥用。

硬限制（hard limit）作为不可突破的边界，可以防止单一进程耗尽系统资源。

软限制（soft limit）则允许进程在安全范围内动态调整资源用量，适应用户不同需求。

```

1 pub enum ResourceLimitType {
2     CPU = RLIMIT_CPU,           // 进程最大CPU时间（秒），超时触发
3     SIGXCPU信号
4     FSIZE = RLIMIT_FSIZE,       // 文件大小上限（字节），超限触发SIGXFSZ
5     信号
6     DATA = RLIMIT_DATA,        // 数据段（堆+静态区）最大内存量
7     STACK = RLIMIT_STACK,       // 主线程栈空间上限，超限触发SIGSEGV
8     CORE = RLIMIT_CORE,         // core dump文件最大尺寸（0=禁用）
9     RSS = RLIMIT_RSS,           // 物理内存驻留集上限
10    NPROC = RLIMIT_NPROC,        // 用户可创建进程/线程总数
11    NOFILE = RLIMIT_NOFILE,      // 文件描述符最大数量
12    MEMLOCK = RLIMIT_MEMLOCK,    // 锁定内存上限（mlock/mlockall）
13    AS = RLIMIT_AS,              // 虚拟地址空间总量限制
14    LOCKS = RLIMIT_LOCKS,        // 文件锁最大数量（fcntl）
15    SIGPENDING = RLIMIT_SIGPENDING, // 待处理信号队列容量
16    MSGQUEUE = RLIMIT_MSGQUEUE,  // POSIX消息队列最大字节数
17    NICE = RLIMIT_NICE,          // nice值调整范围限制（非标准扩展）
18    RTPRIO = RLIMIT_RTPRIO,      // 实时优先级上限
19    RTTIME = RLIMIT_RTTIME,      // 实时任务单次调度时长（微秒）
20 }

```

代码 7-15

```

1 pub struct ResourceLimit {
2     pub soft: u64, // 用户可调整的当前限制
3     pub hard: u64, // 特权级设置的最大允许值
4 }
5
6 pub struct ResourceLimits([ResourceLimit; RLIM_NLIMITS as
7     usize]); // 进程级限制配置表

```

代码 7-16

7.8 进程管理总结

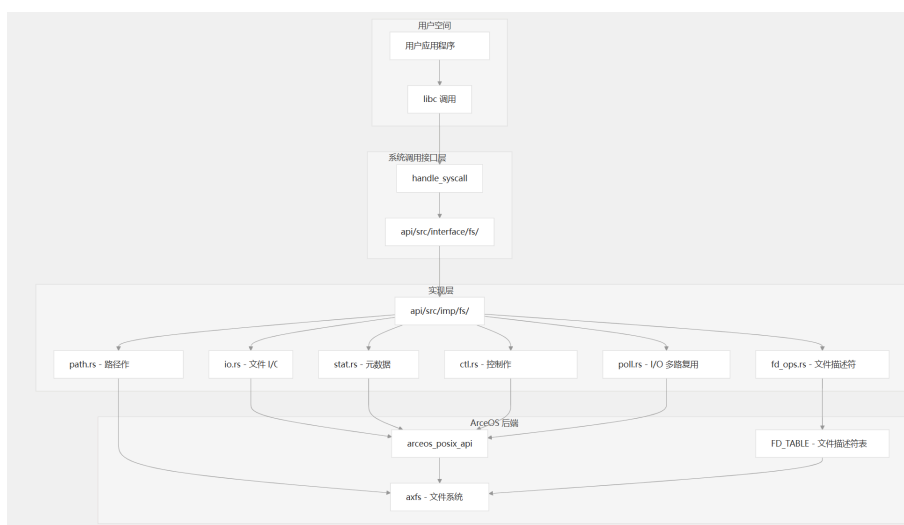
Undefined OS 进程管理模块采用分层架构设计，完整实现了符合 POSIX 标准的高级进程管理体系，主要特性为：

1. 完整的 POSIX 进程/线程生命周期管理
2. 严格的会话-进程组-进程层级结构
3. 安全的父子进程关系维护
4. 僵尸进程和孤儿进程处理
5. 资源限制与隔离机制
6. 操作系统无关的通用接口

第 8 章 文件系统模块

Undefined OS 的文件系统模块构建在一套抽象统一、模块解耦的 VFS 框架之上，支持挂载多个后端文件系统（如 Ext4、fat32、Devfs 等），并通过统一的 Direntry、Inode、File 抽象，协调路径解析、文件访问等操作流程。

下图展示了 Undefined OS 文件系统在内核中的总体架构。



8-1 文件系统总体框架

8.1 虚拟文件系统架构

Undefined OS 实现了分层虚拟文件系统(VFS)架构，该架构通过统一接口抽象不同文件类型，提供一致的系统调用访问。

虚拟文件系统（Virtual File System，简称 VFS）是内核中负责与各种字符流（如磁盘文件，IO 设备等等）对接，并对外提供操作接口的子系统。它为用户程序提供了一个统一的文件和文件系统操作接口，屏蔽了不同文件系统之间的差异和操作细节，使得用户程序使用统一 API 访问不同类型的文件存储介质。

这意味着，用户程序可以使用标准的系统调用，如 `read()`、`write()` 来操作文件，而无需关心文件实际存储在哪种类型的文件系统或存储介质上。

具体实现上：

1. 我们统一了接口抽象，通过 `FileLike trait` 定义了所有文件类型的标准操作接口，这样无论是磁盘文件、管道还是设备文件，用户都可以使用相同的 `read/write` 等

系统调用进行操作。这种设计使得应用程序无需关心底层存储介质差异，且新增文件类型只需实现标准接口即可自动融入系统，系统调用处理流程也得到了得到极大简化。

2. 在 VFS 层之下，我们实现了可插拔的后端文件系统架构。当前系统已支持 Ext4、fat32、Devfs、Procf 这四种不同的文件架构。
3. 我们目前已支持多种文件类型，包括磁盘文件(基于 axfs_ng 的物理文件系统实现)、管道文件（进程间通信缓冲区）、目录文件（路径解析和元数据管理）、标准 I/O 设备（终端输入输出）

```

1 // 全局文件描述符表(进程共享)
2 pub static FD_TABLE: Lazy<FlattenObjects<Arc<dyn FileLike>>> = ...;
3
4 // 文件抽象接口(支持四类文件)
5 pub trait FileLike: Send + Sync {
6     fn read(&self, buf: &mut [u8]) -> LinuxResult<usize>; // 读取数据
7     fn write(&self, buf: &[u8]) -> LinuxResult<usize>; // 写入数据
8     fn stat(&self) -> LinuxResult<ctypes::stat>; // 获取元数据
9     fn poll(&self) -> LinuxResult<PollState>; // 查询I/O状态
10    // ...其他标准方法
11 }

```

代码 8-1

在 Undefined OS 的设计中，文件系统架构严格遵循 linux 的抽象理念，通过多层次的精妙抽象实现了各类文件系统的统一管理。整个文件系统栈由四大核心数据结构构成有机整体，这些组件协同工作实现了从用户空间系统调用到底层存储介质的完整处理流程：

8.2 文件系统抽象层

在 Undefined OS 中，FatFilesystemInner 和 Ext4Filesystem 结构体承载着文件系统的元数据。

这些结构体不仅封装了 fatfs::FileSystem 或 lwext4_rust::Ext4Filesystem 这样的文件系统核心引擎，更创新性地集成了 inode 分配器这一关键组件。

当文件系统被挂载时，这些对象不仅从存储介质读取元数据构建内存镜像，还通过 stat_fs 方法向 VFS 层提供文件系统的容量、块大小等关键参数。

而 sync_fs 方法则确保内存中的修改能精准同步回存储设备，平衡了性能与数据一致性的关系。


```

1 pub struct FatFilesystemInner {
2     pub inner: ff::FileSystem,           // FAT文件系统核心引擎
3     inode_allocator: Slab<()>,          // inode分配器
4     _pinned: PhantomPinned,              // 防止自引用结构被移动
5 }
6
7 pub struct Ext4Filesystem<M> {
8     inner: Mutex<M, LwExt4Filesystem>, // 线程安全的ext4文件系统实例
9     root_dir: OnceCell<DirEntry<M>>,   // 根目录缓存
10 }

```

代码 8-2

8.3 索引节点抽象层

在文件系统设计中，索引节点（inode）作为核心元数据结构，承担着唯一标识与元数据管理、标准化接口实现、跨文件系统元数据转换等关键职责

Undefined OS 定义 `FatFileNode` 和 `FatDirNode` 作为 FAT32 文件系统的索引节点，`Inode` 作为 Ext4 文件系统的索引节点。这些结构体不仅携带 inode 编号，更通过实现 `FileNodeOps` 和 `DirNodeOps` trait 定义了文件的数据读写、元数据获取、空间分配等行为。

```

1 // ext4文件节点实现
2 impl<M: RawMutex + Send + Sync + 'static> FileNodeOps<M> for
3 Inode<M> {
4     fn read_at(&self, buf: &mut [u8], offset: u64) ->
5     VfsResult<usize> {
6         self.fs.lock()
7         .read_at(self.ino, buf, offset) // 直接调用ext4的读接口
8         .map_err(into_vfs_err)
9     }
10
11     fn set_len(&self, len: u64) -> VfsResult<()> {
12         // 文件截断
13         self.fs.lock().set_len(self.ino, len)?;
14         Ok(())
15     }
16 }

```

代码 8-3

```

1 // FAT文件节点实现
2 pub struct FatFileNode<M: RawMutex + 'static> {
3     fs: Arc<FatFilesystem<M>>, // 所属文件系统
4     inner: FsRef<ff::File<'static>>, // 文件句柄
5     inode: u64, // inode编号
6 }
7
8 // FAT32元数据转换
9 pub fn file_metadata(fs: &FatFilesystemInner, file: &ff::File,
10 node_type: NodeType) -> Metadata {
11     // DOS时间转UNIX时间戳
12     let mtime = dos_to_unix(file.modified());
13
14     Metadata {
15         inode: 1, // FAT32没有原生inode, 模拟实现
16         size: file.size().unwrap_or(0) as u64,
17         atime: dos_to_unix(fatfs::DateTime::new(file.accessed(),
18 fatfs::Time::new(0,0,0,0))),
19         mtime,
20         ctime: dos_to_unix(file.created()),
21         // ...其他字段
22     }
23 }

```

代码 8-4

8.4 目录项抽象层

目录项是管理文件在目录树中的信息的结构体，是对文件路径的抽象。在文件系统中，以挂载点(即文件系统的根目录)为根节点，按照文件夹与下属文件的父子关系逐级向下，形成一个目录树的结构。目录树的每个节点对应一个目录项，每一个目录项都指向一个文件的索引节点。

Undefined OS 通过 `DirEntry` 结构体实现目录项抽象。

Undefined OS 采用 `DirEntry`（目录项）与 `Inode`（索引节点）分离的架构。这种设计实现了路径名与文件实体的解耦，支持硬链接等高级特性，同时通过两级缓存机制优化性能。

`DirEntry` 作为路径与 `Inode` 的中间层，维护多对一映射关系，确保了扩展性。

```

1 // FAT32目录项创建
2 impl<M: RawMutex + Send + Sync + 'static> FatDirNode<M> {
3     fn create_entry(&self, entry: ff::DirEntry, name: impl
4         Into<String>, inode: u64) -> DirEntry<M> {
5         let reference = Reference::new(self.this.upgrade(),
6             name.into());
7         if entry.is_file() {
8             // 创建文件目录项
9             DirEntry::new_file(
10                 FatFileNode::new(self.fs.clone(), entry.to_file(),
11                     inode),
12                 NodeType::RegularFile,
13                 reference,
14             )
15         } else {
16             // 创建目录目录项
17             DirEntry::new_dir(
18                 |this| FatDirNode::new(self.fs.clone(),
19                     entry.to_dir(), inode, this),
20                 reference,
21             )
22         }
23     }
24 }

```

代码 8-5

```

1 // ext4目录查找
2 impl<M: RawMutex + Send + Sync + 'static> DirNodeOps<M> for
3 Inode<M> {
4     fn lookup(&self, name: &str) -> VfsResult<DirEntry<M>> {
5         let mut fs = self.fs.lock();
6         let mut result = fs.lookup(self.ino, name)?; // 在ext4中查
7             找目录项
8         let entry = result.entry();
9         // 创建目录项对象
10        Ok(self.create_entry(&entry, name))
11    }
12 }

```

代码 8-6

8.5 文件对象抽象层

文件对象是进程已打开的文件在内存中的表示，也是进程与文件系统交互的接口，在 Undefined OS 中，File 结构体和 FileNodeOps 实现构成了这个接口。

比如当用户空间发起 `open` 系统调用时，`open_file_like` 函数就会根据路径创建出定制化的文件对象。这些对象不仅封装了底层存储操作，还实现了文件位置的完善管理。

```

1  // 文件打开函数
2  pub fn open_file_like(path: &str, options: Option<OpenOptions>) ->
    LinuxResult<Arc<dyn FileLike>> {
3      let options = options.unwrap_or_else(|| {
4          let mut options = OpenOptions::new();
5          options.read(true); // 默认只读打开
6          options
7      });
8
9      // 尝试作为文件打开
10     if let Ok(file_inner) = fops::File::open(path, &options) {
11         Ok(Arc::new(File::new(file_inner, path)))
12     }
13     // 尝试作为目录打开
14     else if let Ok(dir_inner) = fops::Directory::open_dir(path,
15         &options) {
16         Ok(Arc::new(Directory::new(dir_inner, path)))
17     } else {
18         Err(LinuxError::ENOENT) // 文件不存在
19     }

```

代码 8-7

8.6 进程间通信管道

管道是进程间的通信通道，允许一个进程将数据流输出到另一个进程。

在 Undefined OS 中，Pipe 对象通过 `fd_open` 返回后，在 `sys_poll_impl` 中有独特行为：当写端关闭时，读端会触发 `POLLHUP` 事件，而读写操作中的阻塞行为通过 `task_yield_interreptable` 提供可被信号打断的支持。

```
1 pub fn sys_poll_impl(fds: &mut [PollEntry], timeout: u64, block:
2 bool) -> LinuxResult<isize> {
3     // ...
4     loop {
5         let mut updated = false;
6         for entry in fds.iter_mut() {
7             if let Some(pipe) =
8 f.clone()?.into_any().downcast_ref::<Pipe>() {
9                 if pipe.write_end_close() {
10                     entry.results |= PollFlags::POLLHUP;
11                     updated = true;
12                 }
13             }
14         }
15         // ...
16     }
17     Ok(updated_count)
```

代码 8-8

8.7 文件系统总结

Undefined OS 的文件系统模块构建在一套抽象统一、模块解耦的 VFS 框架之上，支持挂载多个后端文件系统（如 Ext4、fat32、Devfs 等），在设计上借鉴了 Linux 架构，通过 Direntry 等抽象结构，协调路径解析、文件访问等操作流程。

我们的文件系统框架强调模块解耦、抽象统一与类型安全，在确保安全性的同时提升了系统可维护性与扩展性。

第 9 章 信号处理模块

信号是 Linux 的一种经典的进程间通信机制，允许内核或其他进程向目标进程/线程发送信号，信号有默认的处理方式，也可以通过用户自定义的信号处理函数来处理。Undefined OS 的信号处理模块设计旨在提供高效、灵活且兼容 POSIX 标准的信号处理机制。

9.1 信号的发送

Undefined OS 采用 `SignalInfo` 结构封装信号元数据，支持三种粒度的信号传递：

1. 线程级传递：精确控制单个线程行为
2. 进程级传递：通知整个进程组
3. 进程组传递：广播式通知（如终端控制）

```

1  // 信号发送接口
2  fn send_signal_thread(tid: Pid, sig: SignalInfo) -> LinuxResult<()>
3  {
4      let thread_data =
5      get_thread_data(tid).ok_or(LinuxError::EPERM)?;
6      thread_data.signal.send_signal(sig); // 写入目标线程的信号队列
7      Ok(())
8  }
9  fn send_signal_process(pid: Pid, sig: SignalInfo) ->
10 LinuxResult<()> {
11     let process_data =
12     get_process_data(pid).ok_or(LinuxError::EPERM)?;
13     process_data.signal.send_signal(sig); // 分发到进程所有线程
14     Ok(())
15 }
16 fn send_signal_process_group(pgid: Pid, sig: SignalInfo) -> usize {
17     let pg = get_process_group(pgid).unwrap();
18     pg.get_processes().iter().map(p
19         send_signal_process(p.get_pid(), sig.clone()).is_ok() as
20     usize
21     ).sum() // 遍历进程组所有成员发送
22 }
```

代码 9-1

9.2 信号处理时机

Undefined OS 在每次系统调用完成准备返回用户态时，都会检查当前进程的信号状态。如果有待处理的信号，内核会在返回前进行处理。

具体来说，内核会检查进程的信号屏蔽字，确定哪些信号是未被阻塞的，然后根据信号的处理方式（默认、忽略或自定义处理）执行相应的操作。

如果用户定义了信号处理函数，内核会保存当前进程的执行上下文，并切换到用户定义的处理函数执行。处理完成后，内核会恢复上下文并继续执行原来的代码。

在从用户的信号处理程序返回时，用户程序会执行映射到用户空间的一段内核“跳板代码”，以从用户态返回到内核态。

9.3 信号处理与退出状态

进程退出后将成为僵尸进程，僵尸进程的回收是通过 `wait4` 等系统调用实现的。父进程通过调用 `wait4` 等系统调用来获取子进程的退出状态和信号信息。Undefined OS 在处理信号时，如果信号的行为中包含退出进程的操作（如 `SIGKILL` 或 `SIGTERM` 等），内核会在处理完信号后将进程标记为已终止状态，并将信号设置在该进程的退出状态中。这样，父进程在调用 `wait4` 时可以获取到子进程的退出状态和信号信息。

第 10 章 网络模块

Undefined OS 的网络模块是基于 `smoltcp` 库（一个嵌入式 TCP/IP 协议栈）实现的，设计目标是提供高效、灵活且易于扩展的网络通信能力。

Undefined OS 的网络模块提供了对 IPv4 和 IPv6 的支持，实现了 POSIX 风格的 TCP/UDP socket API，并且支持 DNS 查询。支持 socket 的基础功能，包括套接字创建、绑定、监听、连接、发送和接收数据等，支持本地(local/unix)套接字和网络套接字。

网络模块的功能主要通过套接字提供，根据 Unix 的一切都是文件的设计理念，网络模块将套接字抽象为文件描述符，提供统一的文件操作接口。内部实现对 `smoltcp` 库的封装，提供了对网络设备的抽象和管理。网络模块支持多种地址族，包括 `AF_INET`、`AF_INET6` 等，能够处理 IPv4 和 IPv6 双协议栈，实现了 TCP 和 UDP 传输协议。

目前已经能够上网并通过 HTTP/HTTPS 协议正常下载网页等，通过了 `iperf`、`netperf` 等网络性能测试工具的验证，并且通过 LTP（Linux Test Project）相关测试套件，验证了网络模块的功能和性能。

10.1 架构设计

10.1.1 底层网络系统

网络架构采用 `smoltcp` 协议栈作为底层适配层，负责 TCP/UDP/IP 层数据包的收发与管理。通过设备抽象化与接口封装，系统实现了对多种网络设备的统一管理与数据包路由分发。

10.1.1.1 LoopbackDev 实现

`LoopbackDev` 实现 `smoltcp::phy::Device` trait，配置 `medium` 为 `Medium::Ip` 或 `Medium::Ethernet`，作为本地回环设备使用。其核心是内部维护的队列缓冲区，实现数据包的“自发自收”机制。


```

1 // 本地环回设备实现结构
2 pub(crate) struct LoopbackDev {
3     pub(crate) queue: VecDeque<Vec<u8>>, // 数据包缓冲队列（先进先出）
4     medium: Medium,                      // 网络媒介类型（IP/以太网）
5 }

```

代码 10-1

当 `smoltcp` 接口发送 IP 包时触发 `transmit` 方法，数据包被推入队列后立即通过 `receive` 方法取出，形成本地闭环。特别支持 TCP SYN 包监听功能，自动创建 socket 用于本地 TCP 服务测试。

10.1.1.2 设备抽象 (Device)

所有网络设备均实现 `smoltcp::phy::Device` trait 定义的收发接口，负责发送与接收数据包,主要包含两类设备：

- **LoopbackDev**: 本地回环设备，数据包直接在本地收发。
- **DeviceWrapper**: 物理网卡设备的封装，负责与底层驱动交互，实现数据包的实际收发。

Device trait 的核心方法如下：

```

1 impl Device for LoopbackDev {
2     // 收/发令牌类型定义
3     type RxToken<'a> = RxTokenScoop;
4     type TxToken<'a> = TxToken<'a>;
5
6     // 获取设备能力参数
7     fn capabilities(&self) -> DeviceCapabilities {
8         // ...
9     }
10
11     /// 接收数据包：取出队列头部数据
12     fn receive(&mut self, _timestamp: Instant) ->
13     Option<(Self::RxToken<'_>, Self::TxToken<'_>)> {
14         // ...
15     }
16
17     /// 发送数据包：提供发送令牌
18     fn transmit(&mut self, _timestamp: Instant) ->
19     Option<Self::TxToken<'_>> {
20         // ...
21     }
22 }

```

代码 10-2

10.1.1.3 监听与连接管理

系统通过 `ListenTable` 全局管理所有 TCP 监听端口及连接队列。每个端口对应一个独立监听条目，内置 SYN 队列，实现三次握手管理,支持 `accept`、`unlisten`、`can_accept` 等操作。

```
1 // TCP 监听端口表结构
2 pub struct ListenTable {
3     // 线程安全存储: 端口->监听条目的映射
4     tcp: Box<[Mutex<Option<Box<ListenTableEntry>>>]>,
5 }
```

代码 10-3

当有新的 TCP SYN 包到达时，`ListenTable` 会自动创建 socket 并加入队列，后续由 `accept` 返回已连接的 socket。

10.1.1.4 地址与路由管理

系统提供 IP 地址与 `SocketAddr` 的转换工具，支持 IPv4/IPv6，便于与标准库和 `smoltcp` 之间的数据结构互转。

10.1.1.5 DNS 查询

网络模块实现了 DNS 查询功能，支持异步轮询和结果转换，便于应用层进行域名解析。

```
1 // 同步DNS查询接口
2 pub fn dns_query(name: &str) -> AxResult<alloc::vec::Vec<IpAddr>> {
3     let socket = DnsSocket::new(); // 创建DNS专用套接字
4     socket.query(name, DnsQueryType::A) // 发起A记录查询
5 }
```

代码 10-4

系统初始化时，会为环回设备（`LoopbackDev`）和每个物理网卡（`DeviceWrapper`）配置相应参数，所有数据包会根据接口和监听表流入不同的设备，实现统一的网络收发和管理。如需补充路由表或更复杂的多网卡路由规则，可在 `ListenTable` 或设备管理层扩展。

10.1.2 TCP

TCP Socket 的实现位于 `axnet::tcp::TcpSocket`。

封装后的 `smoltcp::socket::tcp::Socket` 提供面向流的可靠通信接口，兼容 POSIX 风格 API。

`TcpSocket` 采用原子变量 `AtomicU8` 进行状态管理，确保状态转换的原子性与正确性。每个套接字内置独立状态字段，所有状态转换均通过原子操作实现，保障多线程环境下的操作安全。

状态枚举定义如下：

```
1 const STATE_CLOSED: u8 = 0;    // 关闭状态
2 const STATE_BUSY: u8 = 1;      // 忙状态
3 const STATE_CONNECTING: u8 = 2; // 连接中
4 const STATE_CONNECTED: u8 = 3; // 已连接
5 const STATE_LISTENING: u8 = 4; // 监听中
```

代码 10-5

TCP Socket 调用 `listen` 方法后，将向全局监听表 `LISTEN_TABLE` 注册并开始侦听指定端口，实现标准 TCP 服务端连接流程。

10.1.3 UDP

UDP Socket 实现位于 `axnet::udp::UdpSocket`。

封装 `smoltcp::socket::udp::Socket` 提供无连接的不可靠数据报服务。

核心功能主要包括：

- `bind`：绑定套接字至本地地址
- `connect`：可选设置默认远程地址
- `send/recv`：通过底层 UDP Socket 完成数据收发
- 不同特性支持：例如阻塞/非阻塞模式、TTL 设置、地址复用等 Socket 选项

第 11 章 总结与展望

初赛阶段，Undefined OS 实现了一个功能相对完善的宏内核操作系统，支持了主流的系统架构（RISC-V64, AArch64, x86_64, LoongArch64），并在此基础上构建了多个核心模块，解耦合的设计提供了良好的可移植性。我们提供了 200 余个系统调用，覆盖大部分基础应用的需求，并通过 LTP 测试套件验证了系统调用的兼容性和稳定性。

为了验证我们的内核的实用性，我们使用来自 Ubuntu 24.04.2 LTS 的部分二进制，组建了一个根文件系统，可以支持运行一些常用的命令行工具和应用程序，如 `gcc`，`grep`，`git`，`micropython`，`sqlite3` 等，此外，Tiny C Compiler 可以在我们的操作系统上自举编译出一个 C 语言编译器，`sqlite3` 等程序也可以在我们的操作系统上正确编译，这意味着我们的内核的实用性和兼容性得到了初步验证。

而在初赛结束之后，我们进一步完善了对硬件平台的适配与对应用程序的支持，计划实现对更多开发工具和库的兼容，同时添加了 GUI 支持。未来我们会继续推进内核组件的设计，在实现内核的同时也希望能为开源社区提供更多的通用性强的操作系统组件，同时，我们也会在后续开发中延续组件化、层次化的设计思想，为我们的内核提供更好的功能裁剪/选择支持。

